

Payload Parser & Argument Testing

Navneet Das

14-09-2026

Header

Course: Coding for AI

Author: Navneet Das

Source Code

Here is the Python script for parsing the payload and enforcing strict argument passing:

```
1 """
2 Program : parse_payload.py
3 Purpose : Parses sensor payload packets while enforcing strict
4           positional-only and keyword-only argument passing.
5 Author  : Navneet Das
6 Date    : 14-09-2026
7 Course  : Coding for AI
8 """
9 def parse_payload(raw_packet: list, delimiter: str, /, *,
10                  correction_offset: int = 0) -> tuple:
11     status_tokens = raw_packet[2].split(delimiter)
12     corrected_sensors = [
13         value if "ERROR" in status_tokens else value +
14         correction_offset
15         for value in raw_packet[1][:]
16     ]
17     return raw_packet[0], corrected_sensors
18 #=====main code
19 #=====
20 try:
21     print("\t\t\tPayload Parser & Argument Testing")
22     print("-" * 70)
23     #
24     # Test packets
25     #
26     packet_1 = [501, [12.5, 13.0, 11.8], "NOMINAL|CALIBRATED"]
27     packet_2 = [502, [9.0, 8.5], "ERROR|SENSOR_FAULT"]
28     packet_3 = [503, [1.0, 2.0], "NOMINAL"]
```

```

29     packet_4 = [504, [], "NOMINAL"]
30     packet_5 = [505, [10.0, 20.0, 30.0], "NOMINAL|CALIBRATED|LOW_POWER"
31 ]
32
33     print(f"\n|-{ '-'*50 }-|")
34     print("\n--- Test 1: Normal correction applied ---")
35     result_1 = parse_payload(packet_1, "|", correction_offset=2)
36     print(f"Test 1 -> ID: {result_1[0]} | Sensors: {result_1[1]}")
37     assert packet_1[1] == [12.5, 13.0, 11.8], "Original packet_1
sensors were mutated!"
38     assert result_1[1] is not packet_1[1], "Returned list must be a
different object!"
39     assert id(result_1[1]) != id(packet_1[1]), "Returned list must have
a different id!"
40     print("Test 1 side-effect assertions passed.")
41
42     print("\n--- Test 2: ERROR status present -> no correction applied
---")
43     result_2 = parse_payload(packet_2, "|", correction_offset=5)
44     print(f"Test 2 -> ID: {result_2[0]} | Sensors: {result_2[1]}")
45     assert packet_2[1] == [9.0, 8.5], "Original packet_2 sensors were
mutated!"
46     assert result_2[1] is not packet_2[1], "Returned list must be a
different object!"
47     assert result_2[1] == packet_2[1], "Values should be unchanged when
ERROR is present!"
48     print("Test 2 side-effect assertions passed.")
49
50     print("\n--- Test 3: Default correction_offset (omitted entirely)
---")
51     result_3 = parse_payload(packet_3, "|")
52     print(f"Test 3 (default offset) -> ID: {result_3[0]} | Sensors: {
result_3[1]}")
53     assert result_3[1] == [1.0, 2.0], "Default offset of 0 should leave
values unchanged!"
54
55     print("\n--- Edge case: correction_offset=0 explicit vs omitted ---")
56     result_3b = parse_payload(packet_3, "|", correction_offset=0)
57     print(f"Edge case (offset=0 explicit) -> ID: {result_3b[0]} |
Sensors: {result_3b[1]}")
58     assert result_3b[1] == result_3[1], "Explicit offset=0 must behave
like the default!"
59     print("Edge case passed: offset=0 explicit matches offset omitted."
)
60
61     print("\n--- Edge case: Empty sensor list ---")
62     result_4 = parse_payload(packet_4, "|", correction_offset=3)
63     print(f"Edge case (empty sensor list) -> ID: {result_4[0]} |
Sensors: {result_4[1]}")
64     assert result_4[1] == [], "Empty sensor list should return an empty
list!"
65     print("Edge case passed: empty sensor list handled without error.")
66
67     print("\n--- Edge case: More than two status codes ---")
68     result_5 = parse_payload(packet_5, "|", correction_offset=1)
69     print(f"Edge case (3+ status codes) -> ID: {result_5[0]} | Sensors:
{result_5[1]}")

```

```

69     assert result_5[1] == [11.0, 21.0, 31.0], "Multi-token status
70     parsing failed!"
71     print("Edge case passed: multi-token status flag parsed correctly."
72     )
73
74     print(f"\n|-{ '-'*50 }-|")
75
76     print("\n--- Test 4: Illegal call - positional-only args passed as
77     keywords ---")
78     try:
79         parse_payload(raw_packet=packet_1, delimiter="|")
80         raise AssertionError("Test 4 failed: no TypeError was raised!")
81     except TypeError as e:
82         print(f"Test 4 passed. TypeError raised as expected: {e}")
83
84     print("\n--- Test 5: Illegal call - keyword-only arg passed
85     positionally ---")
86     try:
87         parse_payload(packet_1, "|", 2)
88         raise AssertionError("Test 5 failed: no TypeError was raised!")
89     except TypeError as e:
90         print(f"Test 5 passed. TypeError raised as expected: {e}")
91
92     print(f"\n|-{ '-'*50 }-|")
93
94     except AssertionError as ae:
95         print(f"Assertion Error: {ae}")
96     except Exception as e:
97         print(f"Unexpected Error: {e}")
98     finally:
99         input("\n\n\n\n\nPress Enter to Exit....")

```

Conclusion

Calling `parse_payload` the two illegal ways both failed exactly as intended, since `raw_packet` and `delimiter` are positional-only and `correction_offset` is keyword-only.

Calling `parse_payload(raw_packet=my_packet, delimiter="|")` raised a `TypeError`: *"parse_payload() got some positional-only arguments passed as keyword arguments: 'raw_packet, delimiter'"*, because Python refuses to bind positional-only parameters by name.

Calling `parse_payload(my_packet, "|", 2)` raised a `TypeError`: *"parse_payload() takes 2 positional arguments but 3 were given"*, because `correction_offset` must be supplied as `correction_offset=...` and cannot slide into the third positional slot.

Both errors confirm the signature is doing its job: it makes misuse impossible rather than merely discouraged, catching mistakes immediately at the call site instead of allowing silent, hard-to-trace bugs later in the mission-data pipeline.